

## Tarea 6

### Índice

|                        |   |
|------------------------|---|
| 1. Problema 1 .....    | 1 |
| 1.1. Solución .....    | 1 |
| 1.2. Complejidad ..... | 2 |
| 1.3. Ejecución .....   | 2 |
| 2. Problema 2 .....    | 2 |
| 2.1. Solución .....    | 2 |
| 2.2. Complejidad ..... | 3 |
| 2.3. Ejecución .....   | 3 |
| 3. Problema 3 .....    | 3 |
| 3.1. Solución .....    | 3 |
| 3.2. Complejidad ..... | 4 |
| 3.3. Ejecución .....   | 4 |

### 1. Problema 1

Usando la idea del **pre-proceso de la cadena de caracteres en KMP** (para determinar en cada posición los mayores prefijos que son también sufijos, i.e. el arreglo resets de la clase), escribir un programa que determine el numero de sub-cadenas distintas dentro de una cadena (o sea, el mismo problema que vimos con la función  $z$ ).

Por ejemplo, para el input:

$$s = \text{BLABLA}$$

se espera el output siguiente

$$15$$

Analizar su complejidad.

#### 1.1. Solución

Para solucionar el problema, realizaremos el preprocesamiento de la cadena al revés  $p^{-1}$  de cada prefijo  $p$  de  $s$ . Notemos que esto funciona ya que los sufijos de  $p^{-1}$  son prefijos de la cadena  $p$ . Por ejemplo:

$$\text{BLAB} \rightarrow \text{BALB}$$

Y los sufijos de la palabra al revés son B, BL, BLA, BLAB. Entonces, el arreglo resets que corresponde a  $p^{-1}$  es

$$K_4 = [0, 0, 0, 1],$$

el de LBALB es

$$K_5 = [0, 0, 0, 1, 2],$$

y el de ALBALB

$$K_6 = [0, 0, 0, 1, 2, 3].$$

En este caso, coinciden los de ambas cadenas. Notemos entonces que el valor máximo de cada arreglo corresponde a la longitud máxima de sufijos que ya existe como sub-cadena antes del índice  $i$ . Entonces,

para cada  $i$ , el número de cadenas nuevas que se generan son  $i + 1 - \max(K_{i+1})$ . En el ejemplo BLABLA, tenemos entonces

$$1 + 2 + 3 + (4 - 1) + (5 - 2) + (6 - 3) = 15.$$

Para la implementación, recorremos la cadena caracter por caracter. Realizamos el pre-proceso para cada prefijo al revés, calculamos su máximo, y lo sumamos al resultado final.

## 1.2. Complejidad

La complejidad temporal es  $O(n^2)$ , ya que recorremos toda la cadena, y en cada iteración, realizamos una llamada a `kmpPreprocess`, que es  $O(n)$  en promedio. La complejidad espacial es  $O(n)$ , el tamaño del arreglo `resets`, que es a lo más  $n$ , por lo que lo podemos fijar al tamaño máximo de  $s$  o crearlo dinámicamente.

## 1.3. Ejecución

Compilamos con

```
g++ p1.cpp -o p1
```

Y ejecutamos con

```
./p1
```

El input es simplemente la cadena a la que queremos aplicarle el algoritmo. Por ejemplo,

```
BLABLA
```

tiene como output

```
15
```

## 2. Problema 2

Usando alguno de los algoritmos vistos en las primeras sesiones (en particular, KMP o función  $z$ ), escribir un programa que reciba cadenas de caracteres **S** (hasta  $10^5$  caracteres) y determine el **palíndromo más corto** que se pueda formar agregando caracteres al final de **S**.

Por ejemplo, con **S** siendo la siguiente cadena:

```
ARANAN
```

el palíndromo más corto que se puede formar a partir de **S**, agregando caracteres, es

```
ARANANARA
```

### 2.1. Solución

Primero, encontramos el palíndromo más largo dentro de la cadena  $S$ . En el ejemplo, es ARA. Para hacer esto, construimos la Longest Prefix Suffix de

$$\text{reversed}(S) + | + S$$

Para nuestro ejemplo,

```
NANARA|ARANAN  
[0, 0, 1, 2, 0, 0, 0, 0, 0, 0, 1, 2, 3]
```

Es decir, hemos realizado el siguiente matcheo:

ARANAN  
NANARA

Entonces, el palíndromo más corto es

ARANANARA

## 2.2. Complejidad

Dado que solo necesitamos realizar el pre-proceso 1 vez, e invertimos la cadena 1 vez, la complejidad temporal es  $O(N)$ . La espacial es de  $O(N)$ , ya que hicimos una copia de la cadena original.

## 2.3. Ejecución

Compilamos con

```
g++ p2.cpp -o p2
```

Y ejecutamos con

```
./p2
```

El input es simplemente la cadena a la que queremos aplicarle el algoritmo. Por ejemplo,

ARANAN

tiene como output

ARANANARA

## 3. Problema 3

Resolver el siguiente problema de Codeforces: <https://codeforces.com/problemset/problem/471/D>

### 3.1. Solución

El problema consiste en encontrar las ocurrencias de una figura en otra. Es decir, si una persona tiene una figura hecha de bloques:

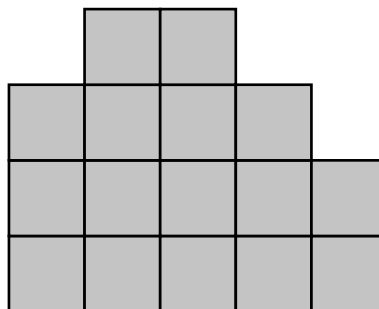


Figura 1: Ejemplo de construcción de bloques representado por 3 4 4 3 2.

Si tenemos otra estructura

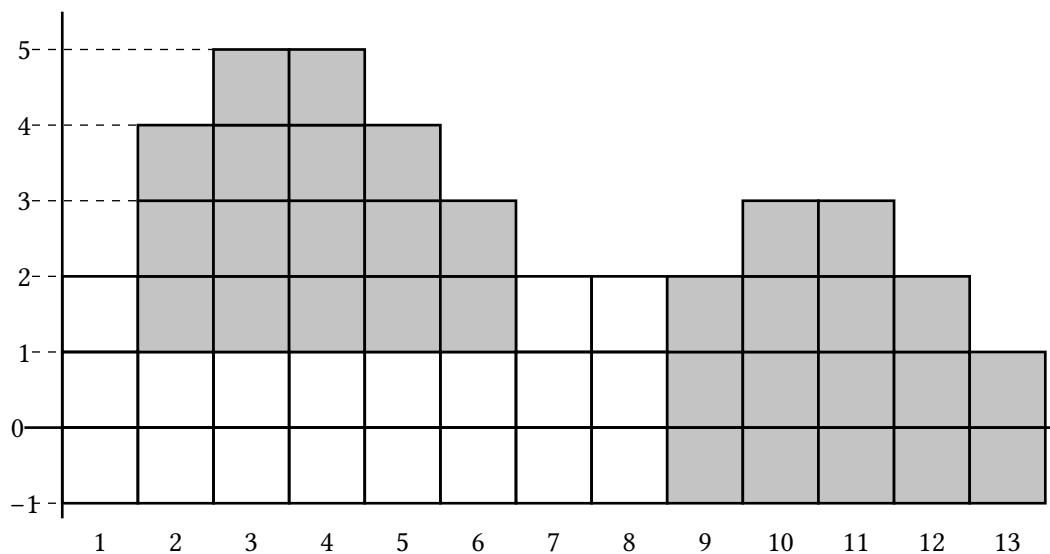


Figura 2: Ejemplo de construcción alterna, donde encontramos 2 patrones similares al de la construcción anterior. Este ejemplo corresponde al input 2 4 5 5 4 3 2 2 2 3 3 2 1.

Podemos mover la estructura original hacia arriba o abajo, como podemos observar en los recuadros sombreados, la estructura original es comienza originalmente en 0, pero *enterramos* la estructura original para que coincidiera con la forma de ésta.

Para solucionar el problema, debemos observar cómo cambian las alturas de las estructuras. Es decir, si la altura de la estructura de longitud  $N$  en cada  $x_i$  es dada por  $h_i$ , queremos comparar los arreglos

$$[h_2 - h_1, \dots, h_{i+1} - h_i, \dots, h_N - h_{N-1}].$$

de cada estructura. Esto funciona porque las coincidencias entre estos arreglos nos dicen que tienen la misma forma de *picos*.

### 3.2. Complejidad

Sea  $m$  la longitud de la primer estructura. Sea  $n$  la de la segunda estructura. El cálculo de los arreglos de diferencias es  $O(n + m)$ . Realizar el pre-proceso es  $O(m)$ . Realizar el algoritmo KMP es  $O(n + m)$ , por lo que la complejidad temporal final es  $O(n + m)$ . La complejidad espacial es  $O(n + m)$ , por los arreglos de diferencias.

### 3.3. Ejecución

Compilamos con

```
g++ p3.cpp -o p3
```

Y ejecutamos con

```
./p3
```

Luego, input esperado es de la forma

```
n m
i_1 ... i_n
j_1 ... j_m
```

Por ejemplo,

```
13 5
2 4 5 5 4 3 2 2 2 3 3 2 1
3 4 4 3 2
```

Genera el output

2